



National University of Singapore

School of Computing

Academic Year 2024/25 Semester 2

LinkedIn Job Recommendation System

BT4221: Advanced Analytics with Big Data Technologies

Group 18's Final Report

Name	Matriculation Number
NG ZHENG LENG BRYAN	A0273387R

Dr. Yeo Wee Kiang

April 17, 2025

Table of Contents

1. Video Presentation	1
2. Project Context & Problem Statement	1
3. Datasets	1
3.1 Job Postings	2
3.2 User Profiles	2
4. Data Preparation & Cleaning	2
4.1 Data Cleaning	2
4.2 Data Preparation	3
4.3 Feature Selection	3
4.4 Handling of Categorical Features	4
5. ML Models	4
5.1 Content-based Filtering	4
5.2 User-based Collaborative Filtering	5
5.3 Hybrid Filtering	5
5.4 Deep Learning Techniques	6
5.4.1 Neural Collaborative Filtering	6
5.4.2 Neural Matrix Factorization	6
6. Model Validation	7
7. Discussion of Results	7
7.1 Content-based Filtering	7
7.2 User-based Collaborative Filtering	8
7.3 Hybrid Filtering	9
7.4 Neural Collaborative Filtering	9
7.5 Neural Matrix Factorisation	10
7.6 Assumptions Made	10
7.7 Challenges Faced	11
7.7.1 High Computational Overhead	11
7.7.2 Lack of Explicit User-Job Interaction Data	11
7.7.3 Delayed Execution Due to PySpark's Lazy Evaluation	11
8. Conclusion	12
Appendix	14
Appendix A: Job Posting Dataset	14
Appendix B: User Profile Dataset	15

1. Project Context & Problem Statement

Job seekers often struggle to identify roles that align with their current skills while lacking visibility into which additional skills could unlock higher-level or transitioning opportunities. This leads to inefficient job searches and missed career growth potential. Traditional job-matching systems often fail to account for both job-specific skills and user profiles holistically, leading to incompatible recommendations. Our project aims to tackle the challenge of building a personalised job recommendation system by leveraging both user profile information and user preferences or behaviours.

We aim to test the hypothesis that deep learning-based models—specifically Neural Collaborative Filtering (NCF) and Neural Matrix Factorization (NeuMF)—can provide more accurate and relevant job recommendations, compared to classical approaches such as content-based filtering (CBF), collaborative filtering, and hybrid methods.

This report evaluates and compares the performance of CBF, NCF and NeuMF using Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Accuracy, and Normalized Discounted Cumulative Gain at rank 5 (NDCG@5), as well as collaborative and hybrid filtering models using Precision@10, Recall@10, Accuracy@10, and F1@10, to assess how well each model captures user-job relevance.

3. Datasets

The raw datasets for this project were sourced from two platforms: Kaggle platform titled **1.3M LinkedIn Jobs & Skills (2024)** and a research article titled **'The Looming Threat of Fake and LLM-generated LinkedIn Profiles'**. Links to the raw datasets and the Google Drive are shown in the table below:

S/N	Dataset Name	Dataset Link	No. of rows	No. of columns
1	1.3M LinkedIn Job Postings	https://www.kaggle.com/datasets/asaniczka/1-3m-linkedin-jobs-and-skills-2024?select=linkedin_job_postings.csv	1,291,224	15
2	3,600 LinkedIn User Profiles	https://github.com/navid-aub/LinkedIn-Dataset?tab=readme-ov-file	3,600	37
3	Google Drive Link (Raw, Processed, Output Data)	https://drive.google.com/drive/folders/1-YdvYRTzhKiU4WBf4O85cC7SWyTjICXE?usp=sharing		

3.1 Job Postings

This dataset captures job posting metadata, comprising 15 columns. The dataset contains a mix of categorical, numerical and boolean features. It includes job-specific details, as well as processing status flags. Among the columns:

- There are 10 categorical columns, mostly representing job metadata (e.g., 'job_title', 'job_location').
- There are five boolean/numerical columns, including identifiers ('job_link_cleaned') and flags ('got_ner', 'is_being_worked').
- The "job_skills" column holds structured text (a list of skills) that will later be parsed for skill-based feature engineering.

Please refer to [Appendix A](#) for full details

3.2 User Profiles

The dataset contains 37 columns and user profile-level information, including personal and professional details from LinkedIn profiles. Among the columns:

- There are 26 categorical columns (e.g. 'Intro', 'Location', 'Skills'), with all columns represented as text
- There are 11 numerical columns (e.g. 'Connections', 'Number of skills', 'Followers'), with all columns represented as integers
- The 'Label' column is a binary variable used to determine if the profile was web-scraped from LinkedIn or generated using Artificial Intelligence.

Please refer to [Appendix B](#) for full details

4. Data Preparation & Cleaning

Since the raw LinkedIn job datasets containing detailed job postings and specific job skills were stored in separate files, merging them was essential to achieve the project objective. The merge was done using an inner join on a primary key: a unique job link ID present in both datasets. The resulting merged dataset, `'linkedin_job_postings_and_skills'`, contains detailed job information with associated skills which will be used in subsequent chapters.

4.1 Data Cleaning

The `'linkedin_dataset'` and `'linkedin_job_postings_and_skills'` datasets contain a wide variety of metadata, some of which is noisy, redundant or irrelevant to the recommendation system.

For the `'linkedin_dataset'`, irrelevant fields such as profile images, timestamps and other unstructured text fields like ('Intro', 'Photo', 'About', 'Label', etc) were removed from the dataset. Next, parse stringified dictionaries in the Skills and Experiences columns into lists of Skills and Experiences. Lastly, typecast the 'Location' and 'Full Name' columns as strings to ensure consistency in these text-based fields.

For the `'linkedin_job_postings_and_skills'` dataset, irrelevant columns like (`'job_link_cleaned'`, `'job_level'`, `'first_seen'`) are removed from the dataset. Next, we focused on cleaning the `job_skills` column, which contained a mix of stringified Python lists, raw strings, and potentially malformed values. We implemented a parsing function that handled all formats, falling back to comma-separated splitting when necessary. This ensured a clean and consistent `job_skills_list` column across all job entries, which was later used to generate Word2Vec embeddings for matching jobs to users based on skill relevance.

4.2 Data Preparation

Since the original dataset lacked explicit user-job interaction records (e.g., clicks, applications, or ratings), we generated pseudo-interactions to simulate user preferences for supervised learning. For each user profile, we computed the cosine similarity between the user's skill vector (generated using Word2Vec embeddings from their listed skills) and the skill vectors of all available job postings.

To mimic realistic positive and negative feedback:

- We selected the top 5 most similar jobs (filtered by location when possible) and labelled them as likes (label = 1), assuming that users would be more interested in jobs closely matching their skill set.
- We also selected the bottom 5 least similar jobs across all locations and labelled them as dislikes (label = 0), simulating jobs that are unlikely to be relevant or preferred.

These generated interactions were stored in separate CSV files (`'liked_list.csv'` and `'dislike_list.csv'`), then exploded and combined into a unified dataset (`'df_combined'`). This allowed us to train binary classification models like NCF and NeuMF, treating the task as a prediction of whether a user would "like" a given job based on historical interaction patterns.

4.3 Feature Selection

In preparing the dataset for training our recommendation models, we carefully selected features that would directly contribute to predicting user-job relevance. For users, we retained the `'Skills'` and `'Experiences'` columns, as these represent the core competencies and work history of a user, which are essential for determining job matches. For jobs, we focused on the `job_skills` column, as it provided the necessary information to compute the similarity between user and job profiles, forming the basis of our recommendation logic. We also kept `'Location'`, `'Full Name'`, and `'job_title'` for encoding and mapping purposes but excluded them from direct prediction in the model.

4.4 Handling of Categorical Features

Several features in our dataset—such as `'Full Name'`, `'Location'`, `'job_title'`, `'company'`, and `'job_location'`—were encoded using `LabelEncoder` because these fields represent categorical

identifiers rather than text with meaningful semantic relationships. By converting them into numerical indices and passing them into embedding layers, we enabled the model to learn dense, trainable vector representations for each unique category. This helped the model capture underlying patterns and associations between categorical features, improving its ability to generalise and make accurate job recommendations.

5. ML Models

Two machine learning models were developed for the LinkedIn Job Recommender System: Neural Collaborative Filtering (NCF), and Neural Matrix Factorization (NeuMF). In the following sub-sections, each model's methodology and rationale for its use in the context of this project will be explained. Results and discussion of each model can be found in [Sections 6](#) and [7](#).

5.1 Deep Learning Techniques

5.2.1 Neural Collaborative Filtering

The NCF model comprises a deep learning architecture, which captures both personalized behaviour and complex user-job interaction patterns.

The embedding layers are responsible for learning latent representations of users and job postings. These layers allow the model to map each unique user and job into a dense vector space where user preferences and job characteristics can be learned directly from interaction data. This enables the model to personalise recommendations for each user, even in the absence of explicit feature data.

The Multilayer component of the model introduces non-linearity into the interaction modelling process. The MLP layers here enable the model to learn complex and conditional relationships, such as a user preferring remote roles only if they are related to data science. Batch normalisation helps stabilise training, while dropout acts as a regularizer to help prevent overfitting.

The NCF model uses Binary-Cross Entropy (BCE) loss with sigmoid activation, which is inherently robust to moderate imbalance. Regularization techniques such as dropout and batch normalisation, along with shuffling during training, further enhance the model's ability to learn meaningful distinctions between liked and disliked job postings—even if future datasets become imbalanced.

5.2.2 Neural Matrix Factorization

To further enhance the quality of personalised job recommendations from the NCF model, we implemented NeuMF, an advanced deep learning architecture that combines the strengths of Generalized Matrix Factorization (GMF) and Multi-Layer Perceptrons (MLP). This hybrid model is capable of learning both linear and non-linear user-job

interaction patterns, making it highly suitable for our task of matching users with jobs based on their implicit preferences.

There are 2 sets of embedding layers, one for the GMF pathway and the other for the MLP pathway. These embeddings map each 'user_id' and 'job_id' to dense vectors that encode latent factors. The GMF embeddings focus on capturing collaborative signals through linear interaction, while the MLP embeddings are optimised for learning non-linear, high-order interactions.

The final layer combines the GMF and MLP outputs into a single feature vector and passes it through a dense layer to produce a prediction logit. This logit is later passed through a sigmoid during training to output the probability that a user would like a particular job.

6. Model Validation

Model	Loss	RMSE	Accuracy	NDCG@5
Neural Collaborative Filtering	Binary Cross Entropy: 0.048	0.1217	97.28%	0.9903
Neural Matrix Factorisation	Mean Square Error: 0.068	0.2611	96.75%	0.977

7. Discussion of Results

7.1 Neural Collaborative Filtering

The NCF model demonstrates strong performance across all evaluation metrics, indicating its effectiveness in both binary classification and personalised recommendation tasks. The **Binary Cross Entropy (BCE) Loss** for the training and testing sets are **0.0422** and **0.0399**, respectively, reflecting the model's well-calibrated confidence in its predictions. These low loss values suggest that the predicted probabilities closely match the true user-item interaction labels.

In terms of **classification accuracy**, the model achieves **97.08%** on the training set and **97.36%** on the test set. This high level of accuracy, combined with a slightly better performance on the test set, highlights that the model generalises well without overfitting. It consistently predicts user preferences correctly, making it highly reliable for real-world applications.

Most notably, the model excels in **ranking quality**, as indicated by its **NDCG@5** scores — **0.9720** for training and an even higher **0.9847** on the test set. NDCG@5 measures the relevance of the top 5 recommended items per user, taking both the position and correctness of each recommendation into account. These near-perfect scores reflect the model's ability to surface the most relevant recommendations at the top of the list, which is critical in practical recommendation systems.

Overall, the NCF model demonstrates high accuracy, low predictive error, and excellent ranking capability, making it a robust and effective solution for personalised job or content recommendation tasks.

7.2 Neural Matrix Factorisation

The Neural Matrix Factorization (NeuMF) model demonstrates strong and reliable performance across all key evaluation metrics, particularly in its ability to generalise to unseen data. The **Binary Cross Entropy (BCE) Loss** is recorded at **0.0512** for the training set and **0.0439** for the test set. These values indicate that the model effectively learns from the training data while maintaining good predictive confidence on the test data, with a notably lower loss during evaluation — a positive sign of generalisation without overfitting.

In terms of **classification accuracy**, the NeuMF model achieves **94.29%** on the training set and **95.11%** on the test set. This high accuracy confirms that the model is able to distinguish between relevant and irrelevant user-item interactions with a high degree of correctness. The increase in accuracy from training to testing also reinforces the model's robustness when applied to new, unseen data.

Importantly, the model performs exceptionally well in terms of **ranking quality**, as reflected by its **NDCG@5** scores of **0.9732** for the training set and **0.9773** for the test set. NDCG@5 is a critical metric for recommendation systems, as it evaluates the relevance of the top 5 recommended items per user while accounting for their ranked positions. The high NDCG@5 scores indicate that NeuMF not only identifies relevant items but also ranks them effectively at the top of the recommendation list.

In summary, the NeuMF model is a robust recommendation engine that delivers high classification accuracy and exceptional ranking quality, with low predictive error. Its ability to perform even better on the test set suggests strong generalisation and reliability, making it well-suited for personalised recommendation scenarios.

7.6 Assumptions Made

A key assumption of the LinkedIn Job Recommendation System is that job seekers prefer job opportunities within or near their geographic location. As such, when evaluating job postings with similar roles or skill requirements, those closer to a user's location will be given a higher similarity score as compared to those further away. Ignoring this consideration may result in less relevant suggestions as users are more likely to disregard job listings that are geographically

impractical. This assumption reflects real-world behaviour, where a job's location may significantly influence a user's decision-making.

7.7 Challenges Faced

7.7.1 High Computational Overhead

One of the main challenges encountered during the development of the LinkedIn Job Recommender System was the lengthy runtime due to the high computational demands of our model. Since our models recommend jobs to users by calculating similarity scores between user profiles and job postings, each user had to be compared against more than one million job listings. This process involved processing millions of rows, leading to significant computational overhead. To address this, we attempted to optimise the data processing pipeline by increasing the number of partitions during computation, limiting the size of processed data during testing phase and PCA to reduce the dimension of sparse user-job matrix. This allowed the workload to be distributed more evenly across resources, thus enabling parallel processing. As a result, we decreased the runtime and improved the recommendation system's efficiency without compromising accuracy.

7.7.2 Lack of Explicit User-Job Interaction Data

Another key challenge was the absence of an explicit user-job interaction matrix. Unlike traditional recommender systems that rely on historical click or application data, our dataset consisted solely of LinkedIn user profiles and job posting information. To overcome this, we had to construct a synthetic interaction matrix by computing cosine similarity scores between user skill embeddings and job skill embeddings. We then assigned interaction labels based on these scores—labeling the top 5 most similar jobs per user as positive interactions (1), and the bottom 5 as negative interactions (0). This workaround allowed us to train a model despite lacking real-world interaction data.

7.7.3 Delayed Execution Due to PySpark's Lazy Evaluation

A further challenge was related to PySpark's lazy evaluation mechanism. In PySpark, transformations such as `.filter()`, `.join()`, and `.withColumn()` are not executed immediately. Instead, they are appended to a logical execution plan and only computed when an action like `.show()`, `.count()`, or `.write()` is triggered. As our pipeline grew longer with many chained transformations, Spark had to recompute the entire sequence of operations every time an action was called. This resulted in significant slowdowns during execution, particularly in the later stages of the pipeline. To address this, we exported the processed DataFrames as CSV files and reloaded them in subsequent steps. This approach helped to break the transformation lineage and reset the computation graph, effectively improving performance without relying on in-memory caching.

8. Conclusion

In this project, crafting pseudo-interactions via **skill-based and user-based similarity** (using Word2Vec/TF-IDF combined with cosine similarity) proved to be a simple yet powerful way to bootstrap training in the absence of real feedback. This strategy enabled our neural models to learn meaningful user–job affinities from the very beginning and offers a scalable blueprint for other domains where interaction logs are sparse or non-existent.

Both **Neural Collaborative Filtering (NCF)** and **NeuMF** consistently delivered strong performance by learning dense, high-dimensional user and job representations. Their ability to capture **non-linear relationships** through deep layers such as multi-layer perceptrons (MLP) allowed the models to uncover subtle, high-order preferences that traditional similarity-based methods cannot.

Our pipeline — from **synthetic label generation** to **embedding-driven neural architectures** — demonstrates a practical framework for solving cold-start recommendation problems. It can be quickly adapted for applications in new marketplaces, niche content platforms, or B2B lead scoring by substituting in domain-specific features.

Possible improvements for our models include:

- **Temporal & Contextual Signals:** Model skill acquisition trajectories, seasonality in hiring, or time-aware decay of preferences.
- **Active Learning Loop:** Gather a small amount of real user feedback to iteratively refine synthetic labels and mitigate labelling biases.
- **Dynamic Hybrid Weighting:** Learn the optimal ' α ' per user or segment, rather than using a fixed global weight, to adapt to varying cold-start severities.
- **A/B Testing in Production:** Validate real-world uplift and calibrate probability outputs against actual application or click-through rates.

These insights underscore how combining creative data engineering (synthetic labels) with modern neural techniques can rapidly bootstrap high-quality recommendation systems, even when historical interactions are virtually non-existent.

References

1. Asaniczka. (2024). *1.3M linkedin jobs & skills (2024)*. Kaggle.
<https://www.kaggle.com/datasets/asaniczka/1-3m-linkedin-jobs-and-skills-2024>
2. Ayooobi, N., Shahriar, S., & Mukherjee, A. (2023). The Looming Threat of Fake and LLM-generated LinkedIn Profiles. Association for Computing Machinery, 1–10.
<https://doi.org/10.1145/3603163.3609064>

Appendix

Appendix A: Job Posting Dataset

Columns	Data Type	Numerical	Categorical
job_link_cleaned	Integer	True	False
last_processed_time	Datetime	False	False
got_summary	Boolean	False	False
got_ner	Boolean	False	False
is_being_worked	Boolean	False	False
job_title	String	False	True
company	String	False	True
job_location	String	False	True
first_seen	Datetime	False	False
search_city	String	False	True
search_country	String	False	True
search_position	String	False	True
job_level	String	False	True
job_type	String	False	True
job_skills	Object	False	False

Appendix B: User Profile Dataset

Columns	Data Type	Numerical	Categorical
Intro	Object	False	True
Full Name	String	False	True
Workplace	String	False	True
Location	String	False	True
Connections	Integer	True	False
Photo	Boolean	False	True
Followers	Integer	True	False
About	String	False	True
Experiences	Object	False	True
Number of Experiences	Integer	True	False
Education	Object	False	True
Number of Educations	Integer	True	False
Licenses	Object	False	True
Number of Licenses	Integer	True	False
Volunteering	Object	False	True
Number of Volunteering	Integer	True	False
Skills	Object	False	True
Number of Skills	Integer	True	False
Recommendations	Object	False	True
Number of Recommendations	Integer	True	False
Projects	Object	False	True
Number of projects	Integer	True	False
Publications	Object	False	True
Number of publications	Integer	True	False

Courses	Object	False	True
Number of Courses	Integer	True	False
Honors	Object	False	True
Number of Honors	Integer	True	False
Scores	Object	False	True
Number of Scores	Integer	True	False
Languages	Object	False	True
Number of Languages	Integer	True	False
Organizations	Object	False	True
Number of Organizations	Integer	True	False
Interests	Object	False	True
Number of interests	Integer	True	False
Activities	Object	False	True
Number of Activities	Integer	True	False
Label	Boolean	False	False